

Now we can use Equation (5.19) to find the steady-state value of K_k :

$$\begin{aligned}
 K_k &= P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1} \\
 &= \frac{P_k^-}{P_k^- + 1} \\
 K_\infty &= \frac{1 + \sqrt{5}}{3 + \sqrt{5}} \\
 &\approx 0.62
 \end{aligned} \tag{5.58}$$

Figure 5.6 shows the *a priori* estimation covariance and the Kalman gain as a function of time, and illustrates their convergence to steady-state values. From the equation for the *a posteriori* estimation covariance, we know that $P_k^+ = (I - K_k H_k) P_k^-$. For this example we therefore see that the steady-state value of P_k^+ is given as

$$\begin{aligned}
 P_\infty^+ &= \left(1 - \frac{1 + \sqrt{5}}{3 + \sqrt{5}} \right) \frac{1 + \sqrt{5}}{2} \\
 &= \frac{1 + \sqrt{5}}{3 + \sqrt{5}}
 \end{aligned} \tag{5.59}$$

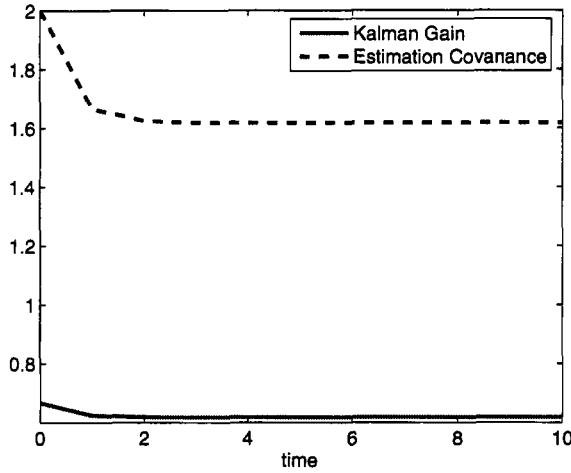


Figure 5.6 Estimation covariance and Kalman gain as a function of time for Example 5.2. The covariance and gain converge to steady-state values.

▽▽▽

5.5 DIVERGENCE ISSUES

The theory presented in this chapter makes the Kalman filter an attractive choice for state estimation. But when a Kalman filter is implemented on a real system it

may not work, even though the theory is correct. Two of the primary causes for the failure of Kalman filtering are finite precision arithmetic and modeling errors [Fit71].

The theory presented in this chapter assumes that the Kalman filter arithmetic is infinite precision. In digital microprocessors the arithmetic is finite precision – only a certain number of bits are used to represent the numbers in the Kalman filter equations. This may cause divergence or even instability in the implementation of the Kalman filter.

The theory presented also assumes that the system model is precisely known. It is assumed that the F , Q , H , and R matrices are exactly known, and it is assumed that the noise sequences $\{w_k\}$ and $\{v_k\}$ are pure white, zero-mean, and completely uncorrelated. If any of these assumptions are violated, as they always are in real implementations, then the Kalman filter assumptions are violated and the theory may not work.

In order to improve filter performance in the face of these realities, the designer can use several strategies:

1. Increase arithmetic precision
2. Use some form of square root filtering
3. Symmetrize P at each time step: $P = (P + P^T)/2$
4. Initialize P appropriately to avoid large changes in P
5. Use a fading-memory filter
6. Use fictitious process noise (especially for estimating “constants”)

These strategies are often problem dependent and need to be explored via simulation or experimentation in order to obtain good results. Some of these strategies may be more attractive than others, depending on the specific problem.

Item 1 above, increasing arithmetic precision, simply forces the digital implementation of the filter to more closely match the analog theory. In a PC-based implementation, it may require only a trivial effort to increase the arithmetic precision – change all the variables to double precision. This trivial change may make the difference between divergence and convergence. However, in a microcontroller implementation it may not be feasible to increase the arithmetic precision.

Item 2 above, square root filtering, is a way of reformulating the filter equations. Even though the physical precision of the implementation does not change, square root filtering effectively increases arithmetic precision. This will be discussed further in Sections 6.3, 6.4, and 8.3. But square root filtering requires more computational effort, which may or may not be a major consideration for a given application. Square root filtering also adds a lot of complication to the filter equations, which invites software bugs.

Items 3 and 4 above involve forcing P to be symmetric and initializing P appropriately. These are easy solutions, but they usually do not result in major improvements to the convergence properties of the filter. However, these steps should always be implemented since they are straightforward and easy, and since they may prevent numerical problems. Note from Equation (5.19) that the P_k^- expression is already symmetric, and so there is no point to forcing symmetry for P_k^- . However, depending on which equation is used, P_k^+ may or may not be symmetric. The

expressions for P_k^+ in Equation (5.19) are mathematically equivalent, but they are not numerically equivalent. One of them has a built-in symmetry, but the others do not. If an equation for P_k^+ is used that does not have a built-in symmetry, then it is very easy and may pay large dividends to force symmetry. This has been done several different ways in the literature. One way is as described in Item 3 above; that is, after P is calculated, set $P = (P + P^T)/2$. Other ways involve forcing the terms below the diagonal to be equal to the terms above the diagonal, or forcing the eigenvalues of P to be positive.

Item 5 above is a simple way of forcing the filter to “forget” measurements in the distant past and place more emphasis on recent measurements. This causes the filter to be more responsive to measurements. It theoretically results in the loss of optimality of the Kalman filter, but it may restore convergence and stability. It is better to have a theoretically suboptimal filter that works rather than a theoretically optimal filter that does not work due to modeling errors. The greater responsiveness of the fading-memory filter to recent measurements makes the filter less sensitive to modeling errors, and hence more robust. This approach will be discussed further in Section 7.4.

Item 6 above, the use of fictitious process noise, is also easy to implement. In fact, it can be implemented in a way that is mathematically equivalent to the fading-memory filter of Item 5. Adding fictitious process noise is a way of telling the filter that you have less confidence in your system model. This causes the filter to place more emphasis on the measurements, and less emphasis on the process model (which may be incorrect) [Jaz69].

■ EXAMPLE 5.3

Let us illustrate the use of fictitious process noise with an example. Suppose we are trying to estimate a state that we think is a constant, but in reality is a ramp. In other words, we have a modeling error. Our assumed (but incorrect) model, upon which we base the Kalman filter, is given as follows:

$$\begin{aligned} x_{k+1} &= x_k + w_k \\ y_k &= x_k + v_k \\ w_k &\sim (0, 0) \\ v_k &\sim (0, 1) \end{aligned} \tag{5.60}$$

The assumed process noise is zero, which means that we are modeling x_k as a constant. From Equation (5.19) we derive the Kalman filter equations for this system as

$$\begin{aligned} P_k^- &= F_{k-1} P_{k-1}^+ F_{k-1}^T + Q_{k-1} \\ &= P_{k-1}^+ \\ K_k &= P_k^- H_k^T (H_k P_k^- H_k^T + R_k)^{-1} \\ &= \frac{P_k^-}{P_k^- + 1} \\ \hat{x}_k^- &= F_{k-1} \hat{x}_{k-1}^+ \\ &= \hat{x}_{k-1}^+ \end{aligned}$$

$$\begin{aligned}
\hat{x}_k^+ &= \hat{x}_k^- + K_k(y_k - H_k \hat{x}_k^-) \\
&= \hat{x}_k^- + K_k(y_k - \hat{x}_k^-) \\
P_k^+ &= (I - K_k H_k) P_k^- (I - K_k H_k)^T + K_k R_k K_k^T \\
&= (1 - K_k)^2 P_k^- + K_k^2
\end{aligned} \tag{5.61}$$

Suppose that the true system, although unknown to the Kalman filter designer, is given as the following two-state model:

$$\begin{aligned}
x_{1,k+1} &= x_{1,k} + x_{2,k} \\
x_{2,k+1} &= x_{2,k} \\
y_k &= x_{1,k} + v_k \\
v_k &\sim (0, 1)
\end{aligned} \tag{5.62}$$

The first state is a ramp, which we assumed incorrectly in our system model to be a constant. Figure 5.7 shows the true state $x_{1,k}$ and the estimated state $\hat{x}_{1,k}$. It can be seen that the estimate is diverging from the true state, and the estimation error is growing without bound.

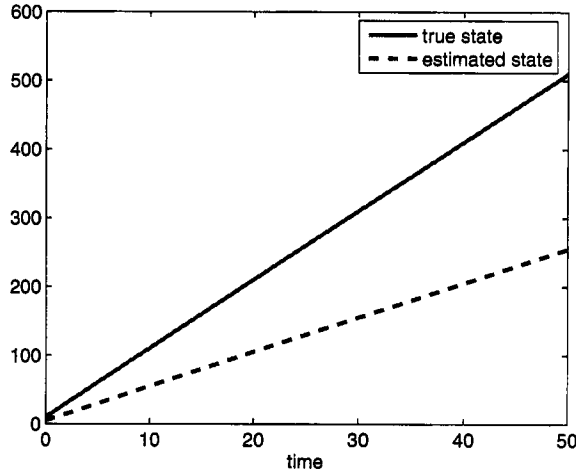


Figure 5.7 Kalman filter divergence due to mismodeling.

However, if we add fictitious process noise to the Kalman filter, then the filter will place more emphasis on the measurements, which will improve the filter performance. Figure 5.8 shows the true state and the estimated state when various values of Q are used in the Kalman filter. As the fictitious process noise gets larger, the estimation error becomes smaller. Of course, this is at the price of poorer performance in case the assumed system model is actually correct. The designer needs to add an appropriate amount of fictitious process noise to balance performance under nominal conditions with performance under mismatch conditions.

Figure 5.9 shows the time history of the Kalman gain K_k for this example for various values of Q . As expected, the gain K_k converges to a larger steady-state value when Q is larger, making the filter more responsive to

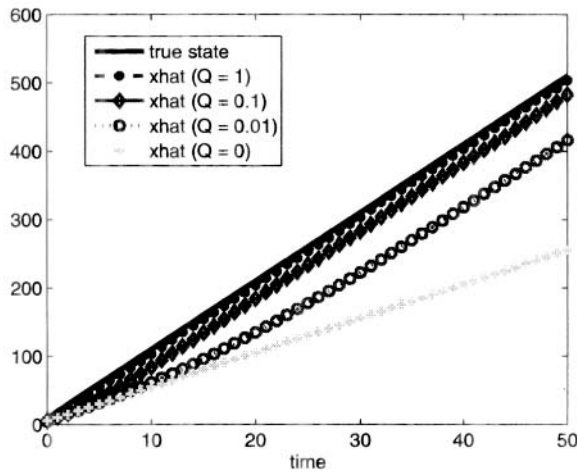


Figure 5.8 Kalman filter improvement due to fictitious process noise.

measurements [see the $\hat{x}_k^+$ expression in Equation (5.61)]. This compensates for modeling errors. As shown later in Section 7.4, the fading-memory filter accomplishes the same thing in a different way. Also note from Figure 5.9 that the steady-state Kalman gain is approximately 0.62 when $Q = 1$. This matches the results of Example 5.2.

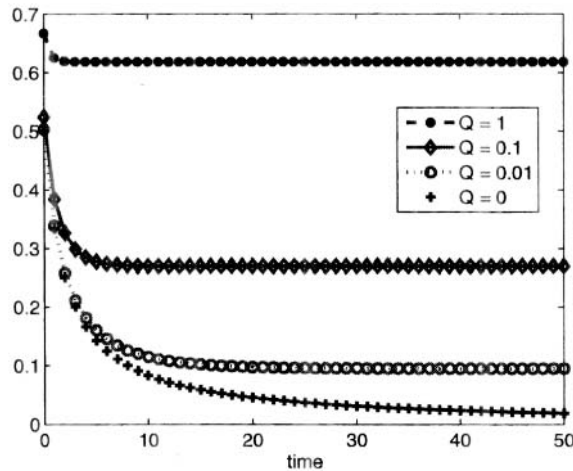


Figure 5.9 Kalman gain for various values of process noise.

This example illustrates the general principle that model noise is good, but only to a certain extent. If a system model has too much noise then it is difficult to estimate its state. But if a system model has too little noise then